Lab 6: Code and Branch Coverage Analysis

Software Testing and Analytics for Projects (SETAP)

Name:	Harshith R Reddy
SRN:	PES2UG24CS188
Section:	D
Repository:	github.com/HarShithRreddy/L6_L7_03_coffee_shop_order

---

Objective

Measure and improve test completeness for the assigned module **coffee_shop_order.py** using Python's **coverage** tool with **pytest**. Additional test cases were written to achieve at least 90% line (code) coverage and at least 90% branch coverage, and the results were analysed from the coverage reports.

Module Under Test

The module contains two functions:

- **process_order_batch(orders, tax_rate)** — computes an order batch total by drink size (small / medium / large), applies syrup and other customizations, enforces allergen-safety and member-discount rules, and returns a status (COMPLETED, NO_SALE, or AUDIT_REJECTED).
- **check_espresso_machine(temperature_cycles)** — scans temperature cycles and returns SHUTDOWN on two consecutive overheat readings, OPTIMAL otherwise, or NO_DATA for empty input.

Test Cases Written

- **CodeCoverageTest_PES2UG24CS188_application.py** — 11 test cases that execute every statement of the module (line coverage).
- **BranchCoverageTest_PES2UG24CS188_application.py** — 17 test cases that additionally exercise the 'false side' of every decision (e.g. strict_allergy = False, an empty orders list, and an empty temperature cycle) to achieve full branch coverage.

Commands Used

```
# Code (line) coverage
coverage run -m pytest CodeCoverageTest_PES2UG24CS188_application.py
coverage report -m coffee_shop_order.py

# Branch coverage
coverage run --branch -m pytest BranchCoverageTest_PES2UG24CS188_application.py
coverage report -m coffee_shop_order.py
```

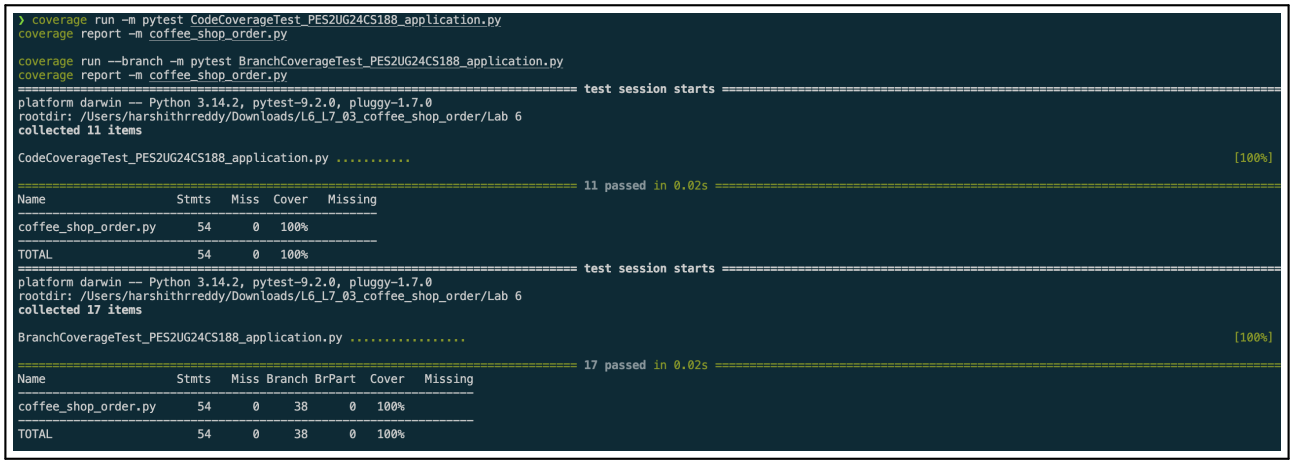
Results Summary

Coverage type	Tests	Statements	Missing	Branches	Coverage

Baseline (starter test)	1	54	28	38 (10 partial)	43%
Line / Code coverage	11	54	0	—	100%
Branch coverage	17	54	0	38 (0 partial)	100%

Both the code-coverage and branch-coverage test suites reach **100%**, exceeding the required 90% threshold. All tests pass.

Terminal Output (Coverage Reports)



Code coverage: 11 passed, coffee_shop_order.py = 100% (0 lines missing). Branch coverage: 17 passed, 38/38 branches, 0 partial = 100%.

Conclusion

Line coverage alone reported 100% while branch coverage initially revealed untested decision paths — demonstrating that line coverage can give a false sense of security. By adding targeted test cases for the untested branches (the false side of each if / elif / else and guard condition), full 100% branch coverage was achieved, ensuring every decision path in the module is exercised.